Fase 2: módulo transversal (health check y system_logs)

Trabajo realizado por el equipo de SmartRoad S.O.S. Fecha: 17/09/2026.

En esta fase nos pidieron agregar a nuestro backend un módulo transversal que sirviera para cualquier sistema: una ruta de salud para saber si el servidor y la base de datos están vivos, y una tabla de trazabilidad para dejar registro de los errores importantes. Aquí explicamos cómo lo dejamos y qué probamos.

1. Endpoint de salud

Creamos una ruta pública (que no pide token) llamada GET /api/health. Cuando alguien la consulta, el backend responde con un JSON que dice si el servidor está bien, cuánto tiempo lleva encendido y el resultado de un ping a la base de datos para confirmar que la conexión sigue viva. Esta respuesta fue la que obtuvimos en la prueba:


```
{
  "ok": true,
  "estado": "OK",
  "servicio": "SmartRoad S.O.S API",
  "version": "1.0.0",
  "uptime": 1962,
  "uptime_human": "0h 32m 42s",
  "db": { "conectado": true, "latencia_ms": 8, "motor": "mysql" },
  "now": "2026-09-17T20:57:17.676Z"
}
```


El campo uptime es el tiempo de actividad del servidor, y el campo db muestra si la base de datos respondió (conectado), cuánto demoró (latencia_ms) y qué motor se está usando. Si la base de datos se llega a caer, la ruta responde con db.conectado en falso y el servidor no se cae; cuando la base vuelve, se reconecta solo sin reiniciar el proceso.

2. Tabla de trazabilidad system_logs

Creamos una tabla independiente llamada system_logs, sin llaves foráneas, con los campos que pedía la actividad:

```
CREATE TABLE IF NOT EXISTS system_logs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  nivel ENUM('INFO','WARNING','ERROR') NOT NULL DEFAULT 'INFO',
  origen_ip VARCHAR(45),
  mensaje VARCHAR(512) NOT NULL
);
```

Los campos son id, fecha_hora, nivel (que solo acepta INFO, WARNING o ERROR), origen_ip y mensaje. La tabla ya está creada en la base de datos smartroad_sos.

3. Registro automático de eventos

Configuramos el backend para que inserte un registro en esta tabla solo, sin que nadie lo tenga que hacer a mano, en dos casos:

Cuando alguien intenta iniciar sesión con una contraseña incorrecta o con un correo que no existe, se guarda un registro de nivel ERROR con la IP desde donde se hizo el intento.

Cuando ocurre un error de conexión o de consulta a la base de datos, también se guarda un registro con el mensaje del error para poder revisarlo después.

Para comprobar que funcionaba hicimos un intento de login con una contraseña equivocada, y esto fue lo que quedó guardado en la tabla:

```
+----+---------------------+-------+-----------+-----------------------------------------------
```

```
-----+
| id | fecha_hora          | nivel | origen_ip | mensaje
    |
+----+---------------------+-------+-----------+-------------------------------------------------
-----+
|  21  |  2026-09-17  15:57:20  |  ERROR  |  ::1           |  [AUTH]  Intento  de  login  fallido  para
"nadie@test.com" |
|    |                     |       |           | (usuario no existe).
    |
+----+---------------------+-------+-----------+-------------------------------------------------
-----+
```

Además dejamos una ruta GET /api/system/logs para consultar los registros, pero restringida solo a usuarios administradores.

4. Archivos que tocamos

El endpoint de salud y la ruta de consulta de logs están en src/server.js. La parte que inserta en system_logs cuando falla una consulta a la base de datos está en src/services/storage.js. El registro de los intentos de login fallidos está en src/routes/auth.js. La tabla está definida en database/schema.sql.

5. Resultado de las pruebas

Probamos la ruta de salud sin token y respondió correctamente con el uptime y el ping a la base de datos. También probamos un login fallido y quedó el registro en system_logs como se muestra arriba. Con esto el módulo transversal queda funcionando.